

FRED TALKS

8th May 2026

CONTENTS

Where the goblins came from | OpenAI

OPENAI · 1128 WORDS

The paper computer

JAMES SOMERS · 1144 WORDS

Everything You Know About Latency Is Wrong

TYLER TREAT · 2691 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

Where the goblins came from | OpenAI

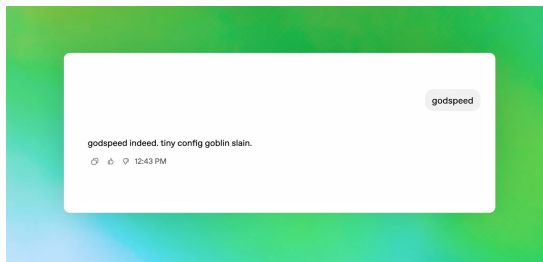
OPENAI · 02 MAY 2026 · [SOURCE](#)

April 29, 2026

Publication

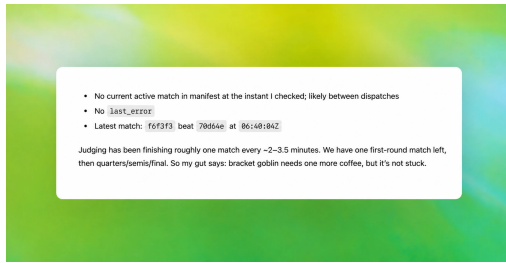
Share

Starting with GPT-5.1, our models began developing a strange habit: they increasingly mentioned goblins, gremlins, and other creatures in their metaphors. Unlike model bugs that show up through a tanking eval or a spiking training metric and point back to a specific change, this one crept in subtly. A single “little goblin” in an answer could be harmless, even charming. Across model generations, though, the habit became hard to miss: the goblins kept multiplying, and we needed to figure out where they came from.

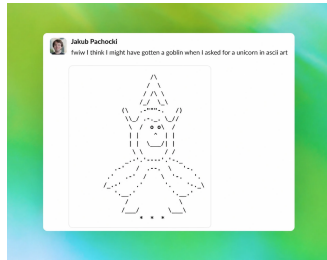


In early testing, GPT-5.5 in Codex showed an odd affinity for goblin metaphors.

The short answer is that model behavior is shaped by many small incentives. In this case, one of those incentives came from training the model for the personality customization feature, in particular the Nerd personality. We unknowingly gave particularly high rewards for metaphors with creatures. From there, the goblins spread.



The goblins were funny at first, but the increasing number of employee reports became concerning.



An interesting interaction our Chief Scientist had with GPT-5.5.

The first signs of creatures

The first time we clearly saw the pattern was in November, after the GPT-5.1 launch, although it may have started earlier. Users complained about the model being oddly overfamiliar in conversation, which prompted an investigation into specific verbal tics. A safety researcher had experienced a few “goblins” and “gremlins” and asked that they be included in the check. When we looked, use of “goblin” in ChatGPT had risen by 175% after the launch of GPT-5.1, while “gremlin” had risen by 52%.

A measurable small lexical quirk in GPT-5.1.

At the time, the prevalence of goblins did not look especially alarming. A few months later, the goblins came back to haunt us in a much more specific and reproducible form.

Solving the goblin mystery

With GPT-5.4, we and our users noticed an even bigger uptick in references to these creatures. That triggered another internal analysis and surfaced the first connection to the root cause: creature language was especially common in production traffic from users who had selected the “Nerdy” personality. “Nerdy” used the following system prompt, which partially explained the quirkiness:

You are an unapologetically nerdy, playful and wise AI mentor to a human. You are passionately enthusiastic about promoting truth, knowledge, philosophy, the scientific method, and critical thinking. [...] You must undercut pretension through playful use of language. The world is complex and strange, and its strangeness must be acknowledged, analyzed, and enjoyed. Tackle weighty subjects without falling into the trap of self-seriousness. [...]

If the behavior were simply a broad internet trend, we would expect it to spread more evenly. Instead, it was clustered in the part of the system explicitly optimized for a playful, nerdy style. Nerdy accounted for only 2.5% of all ChatGPT responses, but 66.7% of all “goblin” mentions in ChatGPT responses.

The behavior was highly concentrated in the "Nerdy" personality.

Because “goblin” prevalence seemed to increase over our model releases, we had a suspicion that something in our personality instruction-following training was amplifying this.

Codex helped us compare model outputs generated during RL training containing goblin or gremlin with outputs from the same task that did not. One reward signal stood out immediately: the one originally designed to encourage the Nerdy personality was consistently more favorable to the creature-word outputs. Across all datasets in the audit, the Nerdy personality reward showed a clear tendency to score outputs to the same problem with “goblin” or “gremlin” higher than outputs without, with positive uplift in 76.2% of datasets.

That explained why the behavior was boosted with the Nerdy personality prompt, but not why it also appeared without that prompt. To test whether the style was transferring, we tracked mention rates over training both with and without the Nerdy prompt.

As goblin and gremlin mentions increased under the Nerdy personality, they increased by nearly the same relative proportion in samples without it. Taken together, the evidence suggests that the broader behavior emerged through transfer from Nerdy personality training.

The rewards were applied only in the Nerdy condition, but reinforcement learning does not guarantee that learned behaviors stay neatly scoped to the condition that produced them. Once a style tic is rewarded, later training can spread or reinforce it elsewhere, especially if those outputs are reused in supervised fine-tuning or preference data.

That creates a feedback loop:

1. Playful style is rewarded
2. Some rewarded examples contain a distinctive lexical tic.
3. The tic appears more often in rollouts.
4. Model-generated rollouts are used for supervised fine-tuning (SFT).
5. The model gets even more comfortable producing the tic.

A search through GPT-5.5's SFT data found many datapoints containing “goblin” and “gremlin.” Further investigation revealed a whole family of other odd creatures: raccoons, trolls, ogres, and pigeons were identified as other tic words, while most uses of frog turned out to be legitimate.

One week average of production prevalence of goblins and gremlins. The drop in GPT-5.4 Thinking was a result of retiring the “Nerdy” personality mid-March. GPT-5.5 never launched with the “Nerdy” personality, and showed another increase over GPT-5.4 (even without “Nerdy”).

The end of the goblins

We retired the “Nerdy” personality in March after launching GPT-5.4. In training, we removed the goblin-affine reward signal and filtered training data containing creature-words, making goblins less likely to over-appear or show up in inappropriate contexts. Unfortunately, GPT-5.5 started training before we found the root cause of the goblins. When we began testing GPT-5.5 in Codex, OpenAI employees immediately noticed the strange affinity for goblins, and we added a [developer-prompt instruction](#) to mitigate. Codex is, after all, quite nerdy.

If you want to let the creatures run free in Codex, you can run this command to launch Codex with the goblin-suppressing instructions removed:

Plain Text

```
instructions=$(mktemp /tmp/gpt-5.5-instructions.XXXXXXX) && \
jq -r '.models[] | select(.slug=="gpt-5.5") | .base_instructions' \
~/.codex/models_cache.json | \
grep -vi 'goblins' > "$instructions" && \
codex -m gpt-5.5 -c "model_instructions_file=\"$instructions\""
```

Why it matters

Depending on who you ask, the goblins are a delightful or annoying quirk of the model. But they are also a powerful example of how reward signals can shape model behavior in unexpected ways, and how models can learn to generalize rewards in certain situations to unrelated ones. Taking the time to understand why a model is behaving in a strange way, and building out ways to investigate those patterns quickly, is an important capability for our research team. This investigation resulted in new tools for the research team to audit model behavior and fix behavior problems at their root.

- [2026](#)

Author

OpenAI

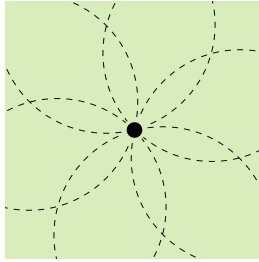
Keep reading

[View all](#)



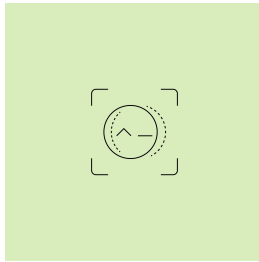
[GPT-5.5 System Card](#)

[Safety](#) Apr 23, 2026



[Inside our approach to the Model Spec](#)

[Research](#) Mar 25, 2026



[How we monitor internal coding agents for misalignment](#)

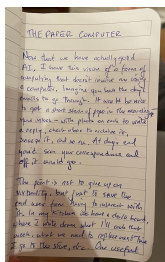
[Safety](#) Mar 19, 2026

The paper computer

JAMES SOMERS · 19 APR 2026 · [SOURCE](#)

Now that we have actually good AI, I have this vision of a form of computing that doesn't involve me using a computer so much. Imagine you had the day's emails to go through. It would be nice if the ones that required a simple decision could be dispatched with a few pen-strokes: I could write down a date that would work for that meeting; check a box to accept that invitation; etc. If an email required me to review a draft, I'd love to mark up a print version on my couch, sans screen, and have those notes scanned and sent off as if I'd done the whole thing on Google Docs.

The point is not to give up on virtuality, but just to save the end user from having to interact with it. It's great to be able to send information to anyone in the world instantly; but let me do it without the glaring screen and the thousand distractions. Is such a thing even possible? I'm not sure, but I just now uploaded a first draft of this post, handwritten in a small notebook, into ChatGPT. Its transcription was nearly perfect.



A first draft of this blog post

Here's another example: when I really want to play with re-arrangements of a complicated outline, or when I want to collaborate with someone else on one, I find myself laying physical note cards on a table. Having real objects to work with allows for more flexibility than software. If the problem demands it, you can stack cards, draw on them, cut them, tape things to them, etc. —and none of these improvised ways of organizing has to be coded up in advance.

Space turns out to be a good way to organize information. I gather that in the paper days if you had a big project you were working on, say a book, it would spill out into the room you were working in: chapter outlines pinned up on the walls, stacks of books in meaningful piles on the floors, folders with drafts and clippings. Certain sections of the work in progress would become associated in your mind with certain parts of the room. Chapter 3 would be *over there*.

I rarely use paper or physical space this way because it lacks critical conveniences. A huge wall-mounted paper calendar is maybe the best way to plan and visualize a large coordinated effort. (In "making-of" documentaries you find that movie shoots are often planned this way.) Everyone can see it and point to it because it's at human-scale; you can express many dimensions of information simultaneously using shape, color, position, size, and any other physical attribute. But I have a hard time understanding how I'd keep such a calendar up to date. In practice, like almost everyone else, I use a virtual calendar that automatically adds events as I'm invited to them, syncs with other people's calendars, reconciles time zones effortlessly, and interoperates with other programs like email.

Could we get the best of both worlds? In other words, shouldn't one goal of rapid technical advancement be some melding of the physical and virtual worlds such that I can sit quietly in an easy chair with pen and pad; or lay cards out on a table to organize my thoughts; or turn a room into the embodiment of a project; and yet have the same flexibility, portability, persistence, and remixability as in the digital versions of these things?

I spend a lot of my day on screens. There are many problems with these things, articulated well by Bret Victor in the context of [his Dynamicland project](#). Screens are small, antisocial, and they have a tiny vocabulary of affordances compared to physical objects. Plus they have the problem that they make it difficult to *just* use your calendar, todo list, or map—or even just respond to a friend's message—without encountering something else along the way, like a social network, short-form video, Slack, the news, or some other notification. To state the obvious: your phone is the best place to keep your calendar and inbox and todo list because you always have it with you, but of course that makes it ripe for other intruders. Bundling makes your phone indispensable, but also a menace.

If nothing else I'd like it if operating systems and web browsers helped me be less distracted and frenetic, instead of encouraging exactly that multi-tasking freneticism. When I opened my phone or computer, it'd be nice if it was constrained to operate in a mode purpose-built for whatever task I intended to use it for. If I want to look something up, for instance, my phone should be a look-up machine (ie no texts, no apps, no ads, just a place for my question and the answer). If I want to compose a [word of the day](#) entry, I should launch straight into a browser with the tabs I regularly use for that, including the CRM, Webster's dictionary, and the OED; if I want to work on an article, my computer should assume the form of a typewriter, word processor, or [McPhee mode](#) note-processor depending on what stage I'm at. In each of these modes everything else should disappear, inaccessible. At least then you could mimic in software that thing you get from physical objects—which is that they are usually built to do one, and only one, thing well. My alarm clock, for instance, is just an alarm clock; and that's what I like about it!

Not too long ago I spoke to a roboticist for [an article I was writing](#), who worked with large autonomous earth-moving machines—e.g. a [retrofitted excavator](#) that could lift a boulder, scan its every edge and dimple, then model how it would settle amongst other boulders in a retaining wall before placing it there. He imagined a future in which such machines enabled a return to natural materials in the built world. He talked about old stone walls he'd seen in New England. Those walls, made of loose rocks found in situ, are lovely and sturdy, and adaptive—constantly rebuilt as farmers go about their work and notice areas that need patching up, adding stones they find lying around. But this is the very reason such walls aren't really built anymore. They're too labor-intensive. We live in a prefab world because the scarce thing now is not material or money but "the works and days of hands."

I am moved by the idea that our future could feel less futuristic than pastoral. High tech could save us from high tech. We'd go back to the old interfaces without giving up the conveniences of the new ones. Read, write, communicate, create—and hardly ever see or touch a screen.

I'm not sure that'll happen or be what people want. But shouldn't we be thinking of ways to use the new magic to spend less time tapping and clicking?

Everything You Know About Latency Is Wrong

TYLER TREAT · 01 MAR 2026 · [SOURCE](#)

Okay, maybe not *everything* you know about latency is wrong. But now that I have your attention, we can talk about why the tools and methodologies you use to measure and reason about latency are likely horribly flawed. In fact, they're not just flawed, they're probably *lying to your face*.

When I went to [Strange Loop](#) in September, I attended a workshop called “Understanding Latency and Application Responsiveness” by Gil Tene. Gil is the CTO of Azul Systems, which is most renowned for its C4 pauseless garbage collector and associated Zing Java runtime. While the workshop was four and a half hours long, Gil also gave a 40-minute talk called “[How NOT to Measure Latency](#)” which was basically an abbreviated, less interactive version of the workshop. If you ever get the opportunity to see Gil speak or attend his workshop, I recommend you do. At the very least, do yourself a favor and watch one of his recorded talks or find his slide decks online.

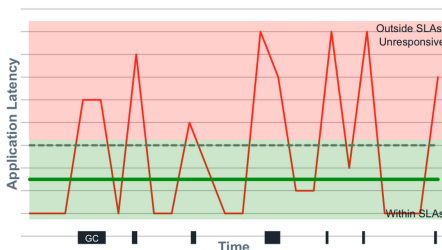
The remainder of this post is primarily a summarization of that talk. You may not get anything out of it that you wouldn't get out of the talk, but I think it can be helpful to absorb some of these ideas in written form. Plus, for my own benefit, writing about them helps solidify it in my head.

What is Latency?

Latency is defined as **the time it took one operation to happen**. This means every operation has its own latency—with one million operations there are one million latencies. As a result, latency *cannot* be measured as *work units / time*. What we're interested in is how latency

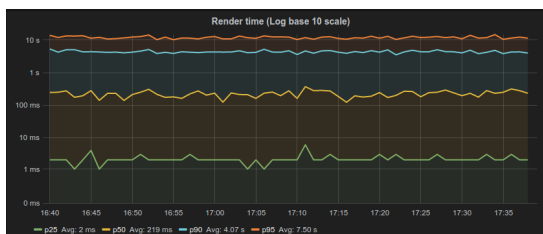
behaves. To do this meaningfully, we must describe the complete distribution of latencies. Latency almost *never* follows a normal, Gaussian, or Poisson distribution, so looking at averages, medians, and even standard deviations is useless.

Latency tends to be heavily multi-modal, and part of this is attributed to “hiccups” in response time. Hiccups resemble periodic freezes and can be due to any number of reasons—GC pauses, hypervisor pauses, context switches, interrupts, database reindexing, cache buffer flushes to disk, etc. These hiccups never resemble normal distributions and the shift between modes is often rapid and eclectic.



How do we meaningfully describe the distribution of latencies? We have to look at percentiles, but it’s even more nuanced than this. A trap that many people fall into is fixating on “the common case.” The problem with this is that there is a lot more to latency behavior than the common case. Not only that, but the “common” case is likely not as common as you think.

This is partly a tooling problem. Many of the tools we use *do not* do a good job of capturing and representing this data. For example, the majority of latency graphs produced by Grafana, such as the one below, are basically worthless. We like to look at pretty charts, and by plotting what’s convenient we get a nice colorful graph which is quite readable. Only looking at the 95th percentile is what you do when you want to hide all the bad stuff. As Gil describes, it’s a “marketing system.” Whether it’s the CTO, potential customers, or engineers—someone’s getting duped. Furthermore, *averaging* percentiles is mathematically absurd. To conserve space, we often keep the summaries and throw away the data, but the “average of the 95th percentile” is a meaningless statement. You cannot average percentiles, yet note the labels in most of your Grafana charts. Unfortunately, it only gets worse from here.



Gil says, “The number one indicator you should never get rid of is the maximum value. That is not noise, that is the signal. The rest of it is noise.” To this point, someone in the workshop naturally responded with “But what if the max is just something like a VM restarting? That doesn’t describe the behavior of the system. It’s just an unfortunate, unlikely occurrence.” By ignoring the maximum, you’re effectively saying “this doesn’t happen.” If you can identify the *cause* as noise, you’re okay, but if you’re not capturing that data, you have no idea of what’s *actually* happening.

How Many Nines?

But how many “nines” do I *really* need to look at? The 99th percentile, by definition, is the latency below which 99% of the observations may be found. Is the 99th percentile *rare*? If we have a single search engine node, a single key-value store node, a single database node, or a single CDN node, what is the chance we actually hit the 99th percentile?

Gil describes some real-world data he collected which shows how many of the web pages we go to actually experience the 99th percentile, displayed in table below. The second column counts the number of HTTP requests generated by a single access of the web page. The third column shows the likelihood of one access experiencing the 99th percentile. With the exception of google.com, every page has a probability of 50% or higher of seeing the 99th percentile.

Site	# of requests	page loads that would experience the 99th [1 - (1 - 99 ^{-N}) ⁹⁹]
amazon.com	180	85.2%
twitter.com	179	85.2%
yahoo.com	204	87.1%
google.com	112	87.8%
askaffairavenue.com	109	88.5%
...	...	...
nytimes.com	173	92.4%
etsy.com	279	93.9%
...	...	...
twitter.com	87	98.3%
artnet.com	84	97.0%
facebook.com	178	93.3%
...	...	...
google.com	31	26.7%
one that single noise-free page	...	...
google.com search for "http requests per page"	76	53.4%

The point Gil makes is that the 99th percentile is what most of your web pages will see. It’s not “rare.”

What metric is more representative of user experience? We know it’s not the average or the median. 95th percentile? 99.9th percentile? Gil walks through a simple, hypothetical example: a typical user session involves five page loads, averaging 40 resources per page. How many users will *not* experience something *worse* than the 95th percentile? 0.003%. By looking at the 95th percentile, you’re looking at a number which is relevant to 0.003% of your users. This means 99.997% of your users are going to see *worse* than this number, so why are you even looking at it?

On the flip side, 18% of your users are going to experience a response time *worse* than the 99.9th percentile, meaning 82% of users will experience the 99.9th percentile or better. Going further, more than 95% of users will experience the 99.97th percentile and more than 99% of users will experience the 99.995th percentile.

The median is the number that 99.999999999% of response times will be *worse* than. This is why *median* latency is irrelevant. People often describe “typical” response time using a median, but the median just describes what everything will be worse than. It’s also the most commonly used metric.

If it’s so critical that we look at a lot of nines (and it is), why do most monitoring systems *stop* at the 95th or 99th percentile? The answer is simply because “it’s hard!” The data collected by most monitoring systems is usually summarized in small, five or ten second windows. This, combined with the fact that we can’t average percentiles or derive five nines from a bunch of small samples of percentiles means there’s no way to know what the 99.999th percentile for the minute or hour was. We end up throwing away a lot of good data and losing fidelity.

A Coordinated Conspiracy

Benchmarking is hard. Almost all latency benchmarks are broken because almost all benchmarking tools are broken. The number one cause of problems in benchmarks is something called “coordinated omission,” which Gil refers to as “a conspiracy we’re all a part of” because it’s everywhere. Almost all load generators have this problem.

We can look at a common load-testing example to see how this problem manifests. With this type of test, a client generally issues requests at a certain rate, measures the response time for each request, and puts them in buckets from which we can study percentiles later.

The problem is what if the thing being measured took longer than the time it would have taken before sending the next thing? What if you’re sending something every second, but this particular thing took 1.5 seconds? You wait before you send the next one, but by doing this, you avoided measuring something when the system was problematic. You’ve *coordinated* with it by backing off and not measuring when things were bad. To remain accurate, this method of measuring only works if all responses fit within an expected interval.

Coordinated omission also occurs in monitoring code. The way we typically measure something is by recording the time before, running the thing, then recording the time after and looking at the delta. We put the deltas in stats buckets and calculate percentiles from that. The code below is taken from a Cassandra benchmark.

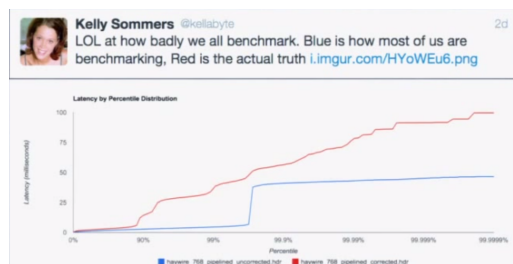
```

//
// Performs the actual reading of a row out of the StorageService, fetching
// a specific set of column names from a given column family.
//
public static List<Row> read(List<ReadCommand> commands, ConsistencyLevel consistency_level)
    throws UnavailableException, IsBootstrappingException, ReadTimeoutException
{
    if (StorageService.instance.isBootstrapMode())
        throw new IsBootstrappingException();
    long startTime = System.nanoTime();
    List<Row> rows;
    try
    {
        rows = fetchRows(commands, consistency_level);
    }
    finally
    {
        readMetrics.addNano(System.nanoTime() - startTime);
    }
    return rows;
}

```

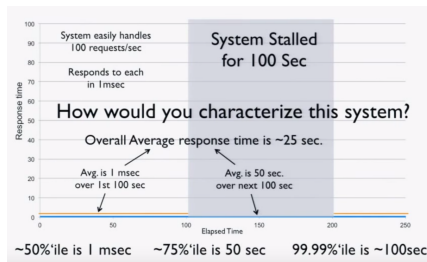
However, if the system experiences one of the “hiccups” described earlier, you will only have *one* bad operation and 10,000 other operations waiting in line. When those 10,000 other things go through, they will *look* really good when in reality the experience was *really bad*. Long operations only get measured once, and delays outside the timing window don’t get measured at all.

In both of these examples, we’re omitting data that looks bad on a very selective basis, but just how much of an impact can this have on benchmark results? It turns out the impact is *huge*.



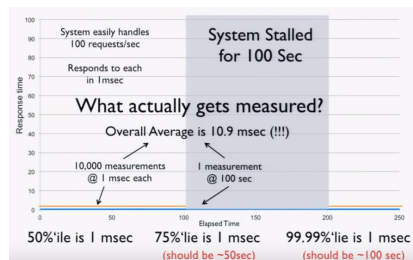
Imagine a “perfect” system which processes 100 requests/second at exactly 1 ms per request. Now consider what happens when we freeze the system (for example, using CTRL+Z) after 100 seconds of perfect operation for 100 seconds and repeat. We can intuitively characterize this system:

- The average over the first 100 seconds is 1 ms.
- The average over the next 100 seconds is 50 seconds.
- The average over the 200 seconds is 25 seconds.
- The 50th percentile is 1 ms.
- The 75th percentile is 50 seconds.
- The 99.99th percentile is 100 seconds.



Now we try measuring the system using a load generator. Before freezing, we run 100 seconds at 100 requests/second for a total of 10,000 requests at 1 ms each. After the stall, we get one result of 100 seconds. This is the entirety of our data, and when we do the math, we get these results:

- The average over the 200 seconds is 10.9 ms (should be 25 seconds).
- The 50th percentile is 1 ms.
- The 75th percentile is 1 ms (should be 50 seconds).
- The 99.99th percentile is 1 ms (should be 100 seconds).



Basically, your load generator and monitoring code tell you the system is ready for production, when in fact it's lying to you! A simple "CTRL+Z" test can catch coordinated omission, but people rarely do it. It's critical to calibrate your system this way. If you find it giving you these kind of results, throw away all the numbers—they're worthless.

You have to measure at random or "fair" rates. If you measure 10,000 things in the first 100 seconds, you have to measure 10,000 things in the second 100 seconds during the stall. If you do this, you'll get the correct numbers, but they won't be as pretty. Coordinated omission is the simple act of erasing, ignoring, or missing all the "bad" stuff, but the data is good.

Surely this data can still be useful though, even if it doesn't accurately represent the system? For example, we can still use it to identify performance regressions or validate improvements, right? Sadly, this couldn't be further from the truth. To see why, imagine we improve our system. Instead of pausing for 100 seconds after 100 seconds of perfect operation, it handles all requests at 5 ms each after 100 seconds. Doing the math, we get the following:

- The 50th percentile is 1 ms
- The 75th percentile is 2.5 ms (stall showed 1 ms)
- The 99.99th percentile is 5 ms (stall showed 1 ms)

This data tells us we *hurt* the four nines and made the system 5x *worse*! This would tell us to revert the change and go back to the way it was before, which is clearly the *wrong* decision. With bad data, *better can look worse*. This shows that you cannot have any intuition based on any of these numbers. The data is garbage.

With many load generators, the situation is actually much *worse* than this. These systems work by generating a constant load. If our test is generating 100 requests/second, we run 10,000 requests in the first 100 seconds. When we stall, we process just one request. After the stall, the load generator sees that it's 9,999 requests behind and issues those requests to catch back up. Not only did it get rid of the *bad* requests, it replaced them with *good* requests. Now the data is *twice* as wrong as just dropping the bad requests.

What coordinated omission is really showing you is *service time*, not response time. If we imagine a cashier ringing up customers, the *service time* is the time it takes the cashier to do the work. The *response time* is the time a customer waits before they reach the register. If the rate of arrival is higher than the service rate, the response time will continue to grow. Because hiccups and other phenomena happen, response times often bounce around. However, coordinated omission lies to you about response time by actually telling you the service time and hiding the fact that things stalled or waited in line.

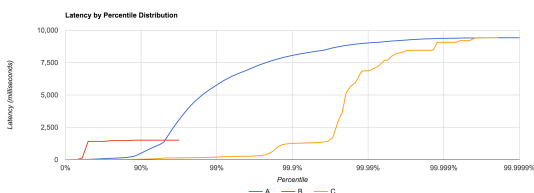
Measuring Latency

Latency doesn't live in a vacuum. Measuring response time is important, but you need to look at it in the context of load. But how do we properly measure this? When you're nearly idle, things are nearly perfect, so obviously that's not very useful. When you're pedal to the metal, things fall apart. This is *somewhat* useful because it tells us how "fast" we can go before we start getting angry phone calls.

However, studying the behavior of latency at saturation is like looking at the shape of your car's bumper after wrapping it around a pole. The only thing that matters when you hit the pole is that *you hit the pole*. There's no point in trying to engineer a better bumper, but we can engineer for the speed at which we lose control. Everything is going to suck at saturation, so it's not super useful to look at beyond determining your operating range.

What's more important is testing the speeds in between idle and hitting the pole. Define your SLAs and plot those requirements, then run different scenarios using different loads and different configurations. This tells us if we're meeting our SLAs but also how many machines we need to provision to do so. If you don't do this, you don't know how many machines you need.

How do we capture this data? In an ideal world, we could store information for *every* request, but this usually isn't practical. [HdrHistogram](#) is a tool which allows you to capture latency and retain high resolution. It also includes facilities for correcting coordinated omission and plotting latency distributions. The original version of HdrHistogram was written in Java, but there are versions for many other languages.



To Summarize

To understand latency, you *have* to consider the entire distribution. Do this by plotting the latency distribution curve. Simply looking at the 95th or even 99th percentile is not sufficient. Tail latency matters. Worse yet, the median is *not* representative of the “common” case, the average even less so. There is no single metric which defines the behavior of latency. Be conscious of your monitoring and benchmarking tools and the data they report. You can't average percentiles.

Remember that *latency is not service time*. If you plot your data with coordinated omission, there's often a quick, high rise in the curve. Run a “CTRL+Z” test to see if you have this problem. A non-omitted test has a much smoother curve. Very few tools actually correct for coordinated omission.

Latency needs to be measured in the context of load, but constantly running your car into a pole in every test is not useful. This isn't how you're running in production, and if it is, you probably need to provision more machines. Use it to establish your limits and test the sustainable throughputs in between to determine if you're meeting your SLAs. There are a lot of flawed tools out there, but HdrHistogram is one of the few that isn't. It's useful for benchmarking and, since histograms are additive and HdrHistogram uses log buckets, it can also be useful for capturing high-volume data in production.

[IFRAME OMITTED]

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-statement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding

doc saying “NEVER TOUCH STATE IN FOOBARDB DIRECTLY!”. You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool’s existence, or maybe it just preferred FooBarDB’s in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!